Lemma 1. *Given a temporal graph $\mathcal{G}_{[t_i,t_j]}$ and a set $C \subset \mathcal{V}_{[t_i,t_j]}$, C must be a CS in $G_{[t_i,t_j]}$ if C is a TCS in $\mathcal{G}_{[t_i,t_j]}$.*

Proof. By the definition of TCS, every ordered pair of vertices in C is connected by a temporal path in $\mathcal{G}_{[t_i,t_j]}[C]$. Removing timestamps gives the corresponding directed paths in $G_{[t_i,t_j]}[C]$. Therefore, C is a CS. □

Lemma 2. *Given a directed graph G and a CS C, C must be a subset of some maximal CS C' in G if C is not a maximal CS.*

Proof. Assume that C is not contained in any maximal CS in G. Let C' be the SCC containing a vertex $u \in C$. Since C' is a maximal CS, there exists a vertex $v \in C \setminus C'$. However, C is a CS, so u and v are mutually reachable in G and must belong to the same SCC. This contradicts $v \notin C'$. Therefore, C is contained in some maximal CS C', and $C \subsetneq C'$ because C is not maximal. □

Lemma 3. *Given a temporal graph $\mathcal{G}_{[t_i,t_j]}$ and two sets $C \subset C' \subseteq \mathcal{V}_{[t_i,t_j]}$, C is not necessarily a TCS when C' is a TCS in $\mathcal{G}_{[t_i,t_j]}$.*

Proof. Assume that every subset of a TCS is also a TCS. Consider $C' = \{u, v, w\}$, where each edge of the directed cycle $u \to v \to w \to u$ occurs at both timestamps 1 and 2. All vertices in C' are mutually temporally reachable, so C' is a TCS. By the assumption, $C = \{u, v\}$ must also be a TCS. However, there is no temporal path from v to u in the induced subgraph on C, contradicting the assumption. Therefore, a subset of a TCS is not necessarily a TCS. □

Lemma 4. *Given a temporal graph $\mathcal{G}_{[t_i,t_j]}$, a TCS $C \subseteq \mathcal{V}_{[t_i,t_j]}$, and a TCS $C' \subseteq \mathcal{V}_{[t_i,t_j]}$, $C \cup C'$ is not necessarily a TCS when $C \cap C' \neq \emptyset$.*

Proof. Assume that the union of any two intersecting TCSs is also a TCS. Consider $C = \{u, v\}$ and $C' = \{v, w\}$, with $u \leftrightarrow v$ at timestamp 2 and $v \leftrightarrow w$ at timestamp 1. Both sets are TCSs and intersect at v. By the assumption, $C \cup C'$ must be a TCS. However, u cannot temporally reach w in their union, since the required timestamps decrease. This contradicts the assumption. Therefore, the union of two intersecting TCSs is not necessarily a TCS. □

Lemma 5. *TCSEnum is* NP*-Complete.*

Proof. We consider whether a temporal graph contains a TCS of size k, where k is part of the input. Given a candidate vertex set, temporal traversals within its induced subgraph can verify mutual temporal reachability in polynomial time. Therefore, the problem belongs to NP.

To prove NP-hardness, we reduce the NP-complete Clique problem to it. Given an undirected graph $G = (V, E)$ and an integer k, construct a temporal graph $\mathcal{G}$ on the same vertex set by replacing each edge $\{u, v\} \in E$ with $(u, v, 1)$ and $(v, u, 1)$. This construction takes polynomial time.

If C is a clique of size k in G, every pair of distinct vertices in C has temporal edges in both directions, so C is a TCS in $\mathcal{G}$. Conversely, if C is a TCS of size k in $\mathcal{G}$, every pair of distinct vertices must be connected directly: all timestamps are 1, so a strictly increasing temporal path cannot contain two or more edges. Thus, every pair is adjacent in G, and C is a clique. Therefore, the decision version of TCSEnum is NP-complete. xx □

Lemma 6. *Given a TCS C of size k in $\mathcal{G}_{[t_i,t_j]}[C_M]$, the induced subgraph $G_R[C]$ must be a clique of size k.*

Proof. Assume that $G_R[C]$ is not a clique. Then there exist two distinct vertices $u, v \in C$ that are not adjacent in G_R. By the definition of G_R, u and v are not mutually temporally reachable in $\mathcal{G}_{[t_i,t_j]}[C_M]$. However, since C is a TCS, they are mutually temporally reachable within C, and hence within C_M. This is a contradiction. Therefore, $G_R[C]$ is a clique of size k. □

Lemma 7. *The time complexity of* Select *is $O(|C_M|(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|))$. The space complexity of* Select *is $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$.*

Proof. Computing all selectivity values by scanning adjacency lists, sorting, and binary search takes $O(|\mathcal{E}^{C_M}_{[t_i,t_j]}| + |C_M| \log |C_M|)$ time. By Lemmas 9 and 10, each traversal takes $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$ time. Each trial performs one forward traversal and at most $|C_M|$ reverse traversals. Including the linear-time SCC computations and pivot selection using stored selectivity values, each trial takes $O(|C_M|(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|))$ time. For a fixed number of trials, the stated time bound follows.

The adjacency lists occupy $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$ space. Timestamp arrays, selectivity values, traversal queues, vertex marks, and SCC working arrays occupy $O(|C_M|)$ space. The SCCs produced by one computation contain at most $|C_M|$ vertices in total and are released after their costs are accumulated. Trials reuse this storage, giving the stated space bound. □

Lemma 8. *Let V be the set of vertices that are retrieved by a temporal traversal starting from a vertex v. Any TCS containing v must be a subset of V.*

Proof. Assume that a TCS C contains v but is not a subset of V. Then there exists a vertex $u \in C \setminus V$. Since C is a TCS, there is a temporal path from v to u within C. Therefore, the temporal traversal starting from v must retrieve u, contradicting $u \notin V$. Thus, $C \subseteq V$. □

Lemma 9. *The time complexity of* FTBFS *is $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. The space complexity of* FTBFS *is $O(|C_M|)$.*

Proof. With temporal edges already sorted by timestamp, initializing the vertex labels takes $O(|C_M|)$ time, and scanning all temporal edges once takes $O(|\mathcal{E}^{C_M}_{[t_i,t_j]}|)$ time. Thus, the total time complexity is $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. The vertex labels, visited marks, and resulting vertex set require $O(|C_M|)$ auxiliary space. □

Lemma 10. *The time complexity of* RTBFS *is* $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. *The space complexity of* RTBFS *is* $O(|C_M|)$.

Proof. Initializing the visited marks takes $O(|C_M|)$ time, and examining each incoming temporal edge at most once takes $O(|\mathcal{E}^{C_M}_{[t_i,t_j]}|)$ time. Thus, the total time complexity is $O(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. The traversal queue, visited marks, and resulting vertex set require $O(|C_M|)$ auxiliary space. □

Lemma 11. *When performing RTBFS in* $\mathcal{G}_{[t_i,t_j]}[C_M]$, *it is safe to filter vertices using* $\tau_{v_p}(\cdot)$.

Proof. Assume that a returned reverse set C_R contains $v \notin C_F$ but misses a vertex u of a TCS containing v. Along a temporal path from u to v, there is an edge (u', v', t) with $u' \notin C_R$ and $v' \in C_R$. Since u' is not retrieved, $\tau_{v_p}(v') \le t$. Thus, v_p can reach v' no later than t and follow the remaining path to v. This contradicts $v \notin C_F$. Therefore, the filtering does not miss such a TCS.xx □

Lemma 12. *Any TCS not fully contained in* C_F *is fully contained in at least one candidate vertex set returned by* RTBFS.

Proof. Assume that a TCS $C \not\subseteq C_F$ is not contained in any reverse candidate set. Choose $u \in C \setminus C_F$. Since all vertices outside C_F are eventually covered, some returned C_R contains u. By Lemma 11, this set contains all vertices of C, contradicting the assumption. Therefore, the lemma holds. □

Lemma 13. *The time complexity of* Decompose *is* $O\left(|T_{C_M}| \cdot |C_M| \left(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|\right)\right)$. *The space complexity of* Decompose *is* $O((h+1)|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. *Here,* $|T_{C_M}|$ *is the number of nodes in* T_{C_M} *and* h *is the height of* T_{C_M}.

Proof. By Lemmas 7, 9, and 10 and the linear-time SCC algorithm, each pivot selection and its associated traversal and SCC computation take at most $O(|C_M|(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|))$ time. These operations are charged to the corresponding nodes of T_{C_M}. The stopping test based on one trial round satisfies the same bound. Therefore, the total time complexity is $O(|T_{C_M}| \cdot |C_M|(|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|))$.

Suppose that the decomposition tree has height h. During depth-first recursion, vertex information is retained for at most $h+1$ levels, each requiring $O(|C_M|)$ space. The temporal edges are shared across recursive calls and stored only once, requiring $O(|\mathcal{E}^{C_M}_{[t_i,t_j]}|)$ space. Therefore, excluding stored output sets, the total space complexity is $O((h+1)|C_M| + |\mathcal{E}^{C_M}_{[t_i,t_j]}|)$. □

Lemma 14. *Let* $\mathcal{R}$ *be the collection of candidate vertex sets returned by* Decompose$(\mathcal{G}_{[t_i,t_j]}, C_M, k)$. *Every TCS of size* k *in* $\mathcal{G}_{[t_i,t_j]}[C_M]$ *is contained in at least one candidate vertex set in* $\mathcal{R}$.

Proof. Assume that a TCS C of size k is not contained in any candidate set in $\mathcal{R}$. At each decomposition, either $C \subseteq C_F$ or C is contained in some C_R, by Lemma 12. By Lemmas 1 and 2, the subsequent SCC decomposition preserves C within a maximal CS of size at least k. Thus, a candidate set containing C is always retained. Further decomposition either stops or produces a strictly smaller candidate containing C. Since C_M is finite, a candidate containing C is eventually returned and included in $\mathcal{R}$, contradicting the assumption. Therefore, every TCS of size k is contained in at least one candidate set in $\mathcal{R}$. □

Lemma 15. *Answer ownership pruning does not miss any TCS of size* k.

Proof. Assume that AOP causes a TCS A of size k to be missed. At each decomposition, let C_j be the first candidate set containing A. For every previously processed candidate C_i, $A \not\subseteq C_i$, so $A \cap H_i \neq \emptyset$, where $H_i = C_M \setminus C_i$. Since $A \subseteq C_j$, restricting H_i to $H_i \cap C_j$ preserves this intersection. Thus, A satisfies every ownership constraint, and C_j is not pruned. Repeating this argument at each decomposition level ensures that A reaches enumeration and verification, contradicting the assumption. Therefore, AOP does not miss any TCS of size k. □

Lemma 16. *For each smaller candidate vertex set* $C_j \subseteq C_M$, *updating its ownership constraints takes* $O(|C_M| + \sum_i |H_i|)$ *time and* $O(|C_M| + \sum_i |H_i|)$ *space, where* $\sum_i |H_i|$ *is the total size of the inherited ownership constraint sets.*

Proof. Marking the vertices of C_j and constructing its complement in C_M take $O(|C_M|)$ time. Each constraint H_i is scanned once, using the marks to compute $H_i \cap C_j$, taking $O(\sum_i |H_i|)$ time in total. The marks and complement occupy $O(|C_M|)$ space, and the restricted constraints occupy at most $O(\sum_i |H_i|)$ space. Therefore, both bounds follow. □

Lemma 17. BPTV *returns* true *if and only if a set* C *is a TCS of size* k.

Proof. Assume that BPTV returns an incorrect result. If it returns true for a non-TCS, all vectors are all-one vectors. However, updates at timestamp t use only vectors for strictly later timestamps, so every set bit corresponds to a valid temporal path. Thus, all vertices are mutually temporally reachable, a contradiction.

If it returns false for a TCS, some vertex u has a zero bit corresponding to another vertex v. Consider a temporal path from u to v. As its edges are processed in decreasing timestamp order, the bit corresponding to v is propagated along the path from its last edge to its first edge. Hence, this bit must be set in u's vector, a contradiction. Therefore, the lemma holds. □

Lemma 18. *For fixed* k, *the time complexity of* BPTV *is* $O(|\mathcal{E}^{C}_{[t_i,t_j]}| \log |\mathcal{E}^{C}_{[t_i,t_j]}|)$. *The auxiliary space complexity is* $O(|\mathcal{E}^{C}_{[t_i,t_j]}|)$.

Proof. Building the priority queue takes $O(|\mathcal{E}^{C}_{[t_i,t_j]}|)$ time. Each edge is extracted at most once, taking $O(\log |\mathcal{E}^{C}_{[t_i,t_j]}|)$ time per extraction. For fixed k, vector updates, copying, and all-one tests take constant time. Thus, the stated time bound follows. The queue occupies $O(|\mathcal{E}^{C}_{[t_i,t_j]}|)$ space. Only

the current and previous vector layers need to be stored, requiring constant space for fixed k. Therefore, the auxiliary space complexity is $O(|\mathcal{E}^{C}_{[t_i,t_j]}|)$. □

Lemma 19. *Every vertex combination directly returned by PIE is a TCS.*

Proof. Assume that PIE directly returns a combination $C[k-1] \cup \{u_k\}$ that is not a TCS. Since $C[k-1]$ is a TCS, any missing temporal reachability must involve u_k. However, $F_{C[k-1]}(u) < t$ ensures that every prefix vertex can temporally reach u_k through (u, u_k, t), while $t' < R_{C[k-1]}(v)$ ensures that u_k can temporally reach every prefix vertex through (u_k, v, t'). Thus, all vertices in the combination are mutually temporally reachable, contradicting the assumption. Therefore, every combination directly returned by PIE is a TCS. □

Lemma 20. *Given sorted timestamp lists for ordered vertex pairs, preprocessing one prefix takes $O(k^3 \log |\mathcal{E}^{C[k-1]}_{[t_i,t_j]}|)$ time, and checking the summary conditions for one additional vertex takes $O(k)$ time. Caching the information for one prefix requires $O(k^2)$ space.*

Proof. Using dense label-setting searches, computing earliest arrival and latest departure labels for one prefix vertex requires $O(k^2)$ timestamp lookups. Each lookup uses binary search and takes $O(\log |\mathcal{E}^{C[k-1]}_{[t_i,t_j]}|)$ time. Repeating this for all prefix vertices and computing F and R gives the stated preprocessing bound. For each additional vertex, checking the two conditions using the maximum and minimum edge timestamps takes constant time per prefix vertex, hence $O(k)$ time in total. The cached label matrices and summary values require $O(k^2)$ space, excluding the input timestamp lists. □